gitctx: Complete Step-by-Step Implementation Plan with Pitfalls, Mitigations, and Verification

This implementation plan defines the construction of **gitctx**, a high-fidelity indexing engine designed to provide **Large Language Models** (LLMs) with verifiable, structured facts from **Git** history. Unlike standard **RAG** (Retrieval-Augmented Generation) systems that index raw text, `gitctx` treats code history as a temporal graph of functional changes.

Project Invariants

To ensure scientific validity and operational reliability, the following invariants are enforced mechanically via CI and runtime checks:

ID	Invariant	Enforcement Mechanism
I1	No network/LLM calls during indexing or retrieval.	`unshare -n` in CI; mock HTTP layer in tests.
I2	Deterministic indexing (Repo + Config = Byte-identical state).	Double-index comparison test (Part 4.2).
I3	Incremental index $\equiv$ Full index.	State equivalence test on commit ranges (Part 4.3).
I4	Every fact carries a verifiable citation.	SQL `NOT NULL` constraints on `commit_sha` and `path`.
I5	No LLM-derived text in embedding inputs.	Physical table isolation for `explain_cache`.
I6	Evaluation corpus is pinned by commit SHA.	`eval/corpus.lock.toml` validation.
I7	Reported numbers include baseline and uncertainty.	Mandatory template columns in `cargo xtask eval`.
I8	Traversal semantics are explicit.	`index_state.traversal` metadata field.

1. Phase −1: Foundation and Corpus Selection

1.1 Repository Scaffolding

- **Build:** A Rust workspace containing `gitctx-core` (extraction), `gitctx-index` (storage), `gitctx-query` (retrieval), and `gitctx-eval`.
- **Pitfalls:** Dependency bloat and slow compile times for the eval harness.
- **Solutions:** Use a shared `core` crate; avoid `tokio` in the indexer if possible to keep it a simple CLI tool.
- **Verification:** `cargo test` passes in under $10\,\mathrm{s}$ for the base workspace.

1.2 Synthetic Fixture Generation

- **Build:** A `tests/fixtures/mkrepo.rs` utility to programmatically generate tiny Git repositories.
- **Pitfalls:** Real-world Git edge cases (evil merges, renames, submodules) are hard to find in small repos.

- **Solutions:** Explicitly generate fixtures for: (1) Linear history, (2) File rename, (3) Cross-file move, (4) Merge with conflict, (5) Squash-merge signature.
- **Verification:** Unit tests for `lineage` must pass against these fixtures with 100% accuracy.

1.3 Corpus Selection and Leakage Stripping

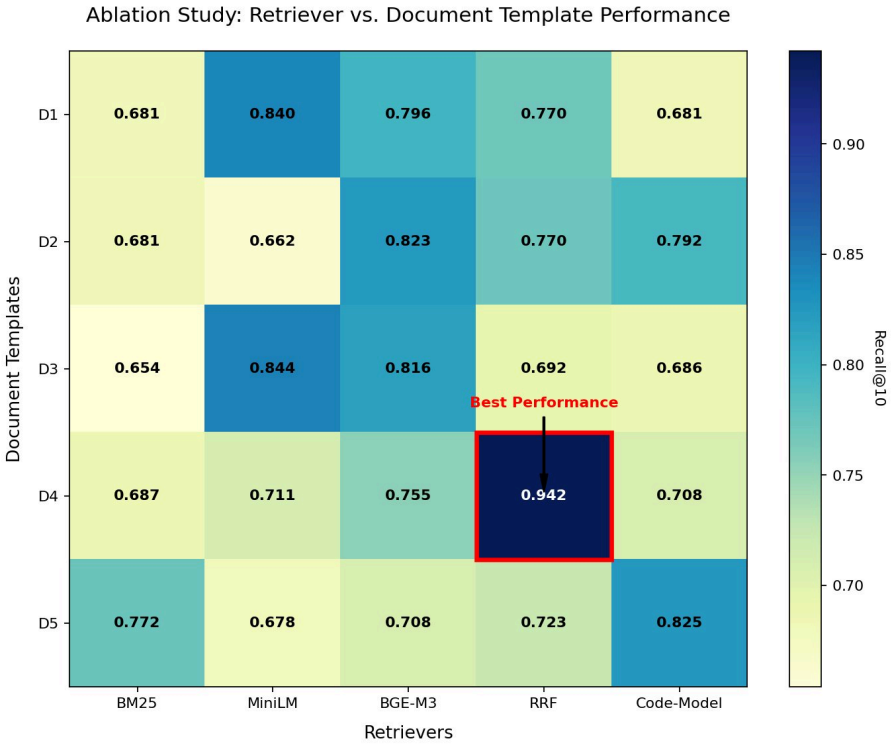
- **Build:** `eval/harvest.py` to select 3-5 Python repos based on issue-linking density (> 150 traceable issues).
- **Pitfalls: Data Leakage** where the query (issue body) contains the answer (commit hash or file name).
- **Solutions:** Strip `Fixes #N`, `Closes`, and PR URLs from the indexed commit messages.
- **Verification:** Grep the index for issue IDs; count must be zero.

2. Phase 0: Python De-risking (The Ablation Grid)

Before committing to the Rust implementation, a Python prototype validates the retrieval thesis.

2.1 The Ablation Grid

- **Build:** A 5×5 matrix testing 5 document templates (D1-D5) against 5 retrievers (R1-R5).
- **Templates:** D1 (Raw Diff), D2 (Commit Message), D3 (Function Signature + Diff), D4 (Templated Pseudo-doc), D5 (D4 + Co-change context).
- **Retrievers:** BM25, MiniLM-L6, BGE-M3, **RRF** (BM25 + MiniLM), and a Code-aware model.
- **Verification:** Calculate **Recall@10** and **MRR** (Mean Reciprocal Rank). A $\Delta < 0.05$ in Recall is treated as noise.



3. Phase 1: Extraction and Function Lineage

3.1 Git Traversal and Historical AST Parsing

- **Build:** A **libgit2** walker using `first-parent` logic. For each commit, parse changed blobs using **tree-sitter**.
- **Pitfalls:** Syntactically broken code in historical commits.
- **Solutions:** Implement a "degraded chunk" fallback that treats the whole file as one chunk if parsing fails.
- **Verification:** Log `parse_success_rate`; target $> 98\%$ for mature repos.

3.2 The Lineage Algorithm

- **Build:** An ordered matching engine to track functions across time.
- **Algorithm Steps:**

1. Exact match (Path + Qualified Name).
2. Move detection (Qualified Name + **MinHash** fingerprint).
3. Rename detection (Path + MinHash).
4. Fuzzy match (MinHash threshold > 0.7).

- **Verification:** Synthetic mutation testing. Apply 10 transformations (M1-M10) to a file and verify the engine tracks the identity.

```rust
// Rust Sketch: Lineage Fingerprinting
pub struct FunctionFingerprint {
    pub qualified_name: String,
    pub minhash: Vec<u64>,
    pub signature_hash: u64,
}

fn calculate_minhash(tokens: &[String]) -> Vec<u64> {
    // Implementation of K-permuted MinHash for Jaccard similarity
    // ...
}
```

4. Phase 2: Indexing and Data Modeling

4.1 SQLite Schema (DDL) The database is the source of truth for all temporal facts.

```sql
-- Core Schema for gitctx
CREATE TABLE index_state (
    schema_version INTEGER PRIMARY KEY,
    config_hash TEXT NOT NULL,
    embedding_model_sha256 TEXT NOT NULL,
    traversal_mode TEXT DEFAULT 'first-parent'
);

CREATE TABLE function_lineage (
    lineage_id INTEGER PRIMARY KEY,
    first_seen_commit TEXT NOT NULL,
    last_seen_commit TEXT NOT NULL,
    canonical_name TEXT NOT NULL
);

CREATE TABLE function_touches (
    lineage_id INTEGER REFERENCES function_lineage(lineage_id),
    commit_hash TEXT NOT NULL,
    path TEXT NOT NULL,
    start_line INTEGER,
```

```
    end_line INTEGER,
    match_confidence REAL,
    PRIMARY KEY (lineage_id, commit_hash)
);

CREATE TABLE call_edges (
    caller_lineage_id INTEGER,
    callee_lineage_id INTEGER,
    valid_from_commit TEXT,
    PRIMARY KEY (caller_lineage_id, callee_lineage_id, valid_from_commit)
);

CREATE TABLE explain_cache (
    fact_hash TEXT PRIMARY KEY,
    llm_explanation TEXT NOT NULL
);
```

4.2 Vector Store and RRF Fusion

- **Build:** A memory-mapped $f32$ vector store for cosine similarity.
- **Fusion:** Combine **Tantivy** (BM25) scores with vector scores using Reciprocal Rank Fusion.
- **Verification:** Determinism test. Index the same repo twice; the `tantivy` segments and SQLite WAL-cleared files must be bit-identical.

5. Phase 3: Agent Surface (CLI & MCP)

5.1 MCP Tool Implementation

- **Build:** A **Model Context Protocol** server exposing tools: `get_function_history`, `find_related_changes`, `query_why`.
- **Pitfalls:** LLMs exceeding token limits with large diffs.
- **Solutions:** Implement a `compact` mode that returns only commit summaries and line ranges, and a `full` mode for specific deep-dives.
- **Verification:** MCP inspector tool validates the JSON-RPC schema.

5.2 Output Envelope Every response must include metadata to ensure the agent knows the context of the facts.

```
{
  "gitctx_version": "0.1.0",
  "config_hash": "a1b2c3d4...",
  "results": [...],
  "warnings": ["Parse error in commit 8f2e1: fallback to raw diff used"]
}
```

6. Evaluation and Risk Management

6.1 Risk Register

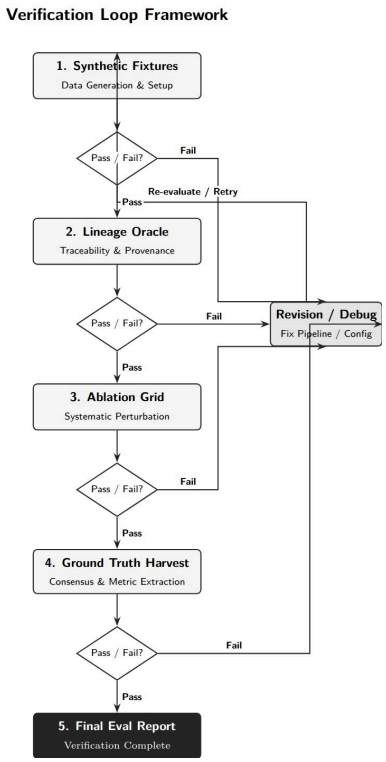
Risk	Likelihood	Impact	Mitigation
Lineage drift in long chains	High	Med	Use **git log -L** as a secondary validation signal.
Embedding model truncation	Med	High	Check truncation rate; use models with $8\ \mathrm{k}+$ context if needed.
SQLite write bottleneck	Low	Med	Single-writer channel pattern with WAL mode enabled.

6.2 Performance Budgets

- **Indexing Speed:** $> 50\,\mathrm{commits} \cdot \mathrm{s}^{-1}$ on a standard workstation.
- **Query Latency:** $< 200\,\mathrm{ms}$ for RRF retrieval.
- **Storage Overhead:** $< 20\%$ of the `.git` directory size.

6.3 Defense and README Framing The project must be framed as a **fact-provider**, not a **reasoner**.

- **Non-Goal:** Generating code or writing commit messages.
- **Goal:** Providing the exact historical context (e.g., “This function was changed 4 times in relation to Issue #402, primarily by Author X”).



Schedule and Cut Order

- **Days 1-5:** Phase −1 (Scaffolding & Fixtures).

- **Days 6-15:** Phase 0 (Python Ablation). **Gate:** If RRF doesn't beat BM25 by 15%, pivot to pure keyword search.
- **Days 16-35:** Phase 1 & 2 (Rust Core & Indexing).
- **Days 36-45:** Phase 3 & 4 (MCP & Final Eval). **Cut Order:** Drop `doc-RAG` first, then drop `call-graph` depth, then drop `blame-attribution`.

This plan ensures that every component of `gitctx` is built on a foundation of measurable accuracy, providing a reliable "memory" for AI-assisted software engineering.